Quantized reward-model verifiers in Best-of-N mathematical answer selection: accuracy, memory and value retention

Hector Borobia[a,*], Elies Seguí-Mas[b], Guillermina Tormo-Carbó[c]

[a] *VRAIN – Valencian Research Institute for Artificial Intelligence, Universitat Politècnica de València, Valencia, Spain*
[b] *Department of Economics and Social Sciences, Universitat Politècnica de València, Valencia, Spain*
[c] *Department of Business Organisation, Universitat Politècnica de València, Valencia, Spain*

Abstract

Best-of-N selection with a learned verifier is a simple way to turn inference compute into accuracy on mathematical reasoning, but the verifier, a reward model of several billion parameters, competes with the generator for accelerator memory. We ask whether post-training quantization of the verifier preserves its selections. On frozen pools of 32 candidates per problem (GSM8K and MATH), we compare three outcome reward models (1.7B, 4B, 8B) and a 7B process reward model in BF16, NF4, GPTQ (4 and 3 bits), AWQ and round-to-nearest, measure memory with packed 4-bit kernels, and fix the analysis plans before the data exist. Four-bit GPTQ leaves the Best-of-32 accuracy of the 8B model unchanged (equivalence within ± 3 points; 90% interval $[-0.6, +1.3]$), and the result replicates in two follow-up studies, with a

*Corresponding author.
Email addresses: hecboar@doctor.upv.es (Hector Borobia), eseguim@upvnet.upv.es (Elies Seguí-Mas), gtormo@omp.upv.es (Guillermina Tormo-Carbó)

7B math generator and with 1,060 new hard problems, for both reward-model types. Where the verifier beats majority vote, 4-bit quantization keeps most of that advantage (the process reward model retains 93% of an 8.2-point advantage). Under a 6 GB verifier budget, a 4B model with packed 4-bit weights beats the best full-precision verifier that fits by 1.4–2.25 points. The mechanism is answer redundancy: quantization changes the selected candidate on a quarter to a third of problems, but the new winner usually carries the same answer; 3-bit weights and the smallest verifier break down. We also show that a strict answer grader can turn reward-model format preferences into spurious quantization gains, and that with a strong generator, majority vote matches or beats reward-model selection on hard problems.
Keywords: post-training quantization, reward models, Best-of-N, test-time compute, mathematical reasoning, preregistration

---

1. Introduction

Sampling several candidate solutions and letting a learned verifier pick one (Best-of-N) is one of the simplest ways to turn extra inference compute into accuracy on mathematical reasoning [1, 2, 3]. The verifier is usually a reward model of a few billion parameters: an outcome reward model (ORM) that scores a complete response, or a process reward model (PRM) that scores each reasoning step. In deployment it competes with the generator for accelerator memory. A natural way to make room is post-training quantization (PTQ) of the verifier's weights to 4 bits or fewer [4, 5, 6].

Whether this is safe is not obvious. PTQ is usually evaluated by perplexity or by the accuracy of the quantized model on its own task, whereas a

verifier only needs to *rank* candidates correctly. Small score perturbations are harmless if they do not reorder the top of the ranking, but Best-of-N amplifies errors at the top: as N grows, the selected candidate is increasingly the one whose score error is largest [7]. One might therefore expect quantization damage to grow with the budget N and with problem difficulty, and to matter most for small verifiers. Conversely, if quantized verifiers select as well as full-precision ones, a memory budget is better spent on a larger quantized verifier than on a smaller full-precision one.

We study these questions in a preregistered, controlled setting. A fixed pool of 32 candidates per problem is generated once and frozen, so that every verifier and every precision ranks exactly the same responses and differences in accuracy can only come from the verifier. We evaluate three ORM sizes and one PRM in BF16, NF4, GPTQ (4 and 3 bits), AWQ and round-to-nearest, on 2,000 GSM8K and MATH problems, and measure memory with packed 4-bit kernels. Five contrasts were registered before any evaluation score was opened; two follow-up studies with a stronger generator and with new problems were registered before their data existed.

Our contributions are:

1. **Equivalence at 4 bits.** Quantizing an 8B ORM with GPTQ-W4 leaves Best-of-32 accuracy unchanged within ± 1.3 pp (registered equivalence test at ± 3 pp), and the result replicates in two preregistered follow-up studies, with a 7B math generator and with the original generator on new hard problems, for both an ORM and a PRM. Three bits is where degradation starts, sharply so for the smallest verifier.
2. **A memory-budget rule.** Under a 6 GB verifier budget, a 4B ORM

with packed 4-bit weights beats the best full-precision verifier that fits by 2.25 pp (registered; 1.4 pp after correcting the strict grader with exhaustive blinded human grading).

3. **A mechanism.** Quantization changes the selected candidate on a quarter to a third of problems at 4 bits, but rarely the correctness of the selected answer, because candidates are redundant: damage is confined to contested problems where competing answers have similar support, and quantization noise tends to move the selection towards the consensus answer.
4. **Value retention.** Where the verifier beats majority vote, 4-bit quantization keeps most of that advantage: in a preregistered study on 1,060 new hard problems the PRM retains 93% of an 8.2 pp advantage (the ORM 91% of 3.0 pp, narrowly significant). With a strong generator, however, majority vote matches or beats reward-model selection on hard problems, which limits how much value a verifier can add in the first place.
5. **Evaluation lessons.** Strict answer graders can turn reward-model format preferences into spurious quantization "gains"; we show how a format/content decomposition, a lenient grader, a blinded human audit and revised labels resolve this, and report two registered hypotheses (damage grows with difficulty and with N) that the evaluation data did not support.

2. Related work

Verifiers and test-time compute.. Training a verifier to rank sampled solutions was introduced for grade-school mathematics by Cobbe et al. [1]. Process supervision, which scores every reasoning step, outperforms outcome supervision on MATH [2], and automatically labelled PRMs [8, 9, 10] and generative verifiers [11] have followed. How to spend inference compute between sampling and verification has become a research question in itself [3, 12, 13]. These works study what verifiers can achieve at full precision; the cost of the verifier itself enters only as a fixed term.

Reward models against self-consistency.. Majority vote over sampled answers (self-consistency) needs no verifier [14], and its coverage and accuracy keep growing with the number of samples [15, 16]. Whether a reward model beats it depends on the generator, the reward model and the aggregation rule [17, 18]. Most relevant to us, Kim et al. [19] compare memory-optimal test-time strategies for reasoning models and find that an external 7B PRM, counted at full precision, is consistently less memory-efficient than majority voting. We complement this result from the verifier side: quantizing the verifier removes most of that memory cost without changing its selections, so the comparison should be made against quantized verifiers.

Reward-model reliability.. Reward-model quality is usually measured on preference and step-error benchmarks [20, 21, 22], and Best-of-N is known to over-optimise imperfect reward models [7]. Our question is different: not whether a reward model is accurate, but whether a compressed copy of it makes the same selections as the original.

Post-training quantization of language models.. Post-training quantization reduces memory with little loss of perplexity or task accuracy at 8 bits [23, 24] and, for weights only, at 4 bits [4, 5, 6], 4-bit precision is close to optimal for a fixed bit budget [25], and large evaluations confirm that well-calibrated 4- and 8-bit models track their full-precision versions [26]; the topic is active in this journal as well [27, 28]. For reasoning, quantization of the *generator* hurts more at 3 bits and on harder problems [29, 30]. Aggregate accuracy can also hide many changed answers: compressed models keep their accuracy while flipping a sizeable share of individual outputs [31]. Our winner-change analysis applies this observation to selection, where a changed winner usually carries the same answer.

Quantized evaluators.. The closest work quantizes LLM-as-judge models: Patira [32] reports that INT8 judges stay within a preregistered equivalence margin of their full-precision agreement with humans, while NF4 flips a quarter to a third of pairwise verdicts on open-ended chat. We study scalar reward models used for Best-of-N selection on problems with verifiable answers, several quantization methods, a memory frontier with packed kernels, and selection value relative to majority vote. To our knowledge, no prior work quantizes the verifier in Best-of-N and measures the effect on selection.

Evaluation practice.. Math benchmark results are sensitive to answer extraction, contamination and label errors [33, 34, 35, 36], and reporting confidence intervals is still uncommon [37]. Preregistration has been proposed for predictive modelling [38] but remains rare in machine learning. We fix the analysis plans before the data exist, report statistical uncertainty for every contrast,

and check the outcome measure with blinded human grading, revised labels and a lenient grader.

3. Experimental setup

All design decisions below were fixed in a public preregistration before any evaluation candidate was scored; amendments were registered before the data they concern existed (Section 3.7). Code, frozen manifests and all scores are released.[TODO: repository URL]

3.1. Problems and candidate pools

The registered evaluation set has 2,000 problems: 800 from the GSM8K test split [1], 400 problems sampled from MATH500 [2], and 800 further MATH test problems [39] outside MATH500, sampled with a fixed seed (text-level exclusion against MATH500 and the development set). A development set of 400 problems (200 GSM8K train, 200 MATH train) was used only for the go/no-go gate and for choices that the preregistration assigns to development data.

The primary generator is Qwen2.5-1.5B-Instruct [40]. For every problem it produces a pool of 32 candidates (4 seeds $\times$ 8 samples, temperature 0.8, top-p 1.0, at most 1,024 new tokens) with the system prompt "Please reason step by step, and put your final answer within `\boxed{}`." Pools are generated once in BF16 with vLLM [41], frozen and hashed; every verifier and precision scores exactly the same candidates. Candidate accuracy is 52.1%, 88.7% of candidates have an extractable final answer, and 6.5% reach the length limit. A secondary generator, Llama-3.2-3B-Instruct, produces pools for a 300-problem subset (exploratory; Section 4.7).

3.2. Verifiers

We use three outcome reward models (ORMs) of one family, Skywork-Reward-V2-Qwen3 at 1.7B, 4B and 8B parameters [42], which score the full (question, response) pair with a scalar head, and one process reward model (PRM), Qwen2.5-Math-PRM-7B [10], whose step rewards are aggregated by their minimum. Both families are run with their documented input formats.[1]

3.3. Precisions and deployments

Weights of all linear layers inside the transformer blocks are quantized; embeddings, norms and the reward head stay in BF16. The registered conditions are:

- **NF4**: 4-bit NormalFloat [6], both simulated and deployed with `bitsandbytes` (bit-identical scores);
- **GPTQ-W4** and **GPTQ-W3**: GPTQ [4] with group size 128, calibrated on 128 WikiText-2 sequences of 512 tokens (generic calibration);
- **AWQ-W4** [5] (same calibration) and round-to-nearest **RTN-W4**/**RTN-W8** (exploratory dose-response).

Accuracy experiments use simulated ("fake") quantization: weights are quantized and dequantized to BF16, so only the rounding changes. For the memory study, GPTQ-W4 models are exported and run with packed 4-bit kernels

[1] Under recent versions of `transformers`, the PRM's remote code leaves its rotary position buffers uninitialised after loading; all PRM results use buffers recomputed as in the model's own initialisation, checked at every load (Appendix A).

(`gptqmodel`, Marlin); packed and simulated GPTQ-W4 select the same final answer on 98.8–99.2% of questions, with symmetric correctness flips (0.35–0.55% of questions). Every quantized model is audited: all target layers must change and no other parameter may change.

Verifier memory is the peak allocated GPU memory (GiB) while scoring a fixed workload of 64 sequences of 900 tokens on an RTX 4090, with no generator resident: 4.0/9.9/17.0 GiB for the 1.7B/4B/8B ORMs in BF16, 2.1/4.9/7.4 GiB with packed GPTQ-W4 and 2.1/5.1/7.7 GiB with NF4.

3.4. Selection rules and baselines

Best-of-N selects the highest-scoring candidate among N; ties are broken by a fixed random priority (seed 11) shared by all conditions. For $N < 32$, accuracy is averaged over 200 random subsets per problem, drawn identically for every condition (seed 7); $N = 32$ is the full pool. Selection-free baselines are unweighted majority vote over canonical answer clusters (numeric normalisation, then symbolic equivalence with `math_verify`; unparseable candidates do not vote), the oracle (pass@32) and a random candidate.

3.5. Grading

The registered grader extracts the last `\boxed{}` answer (else a GSM8K-style "####" line or an explicit "the answer is" phrase), normalises numbers and compares symbolically for MATH. Because a strict grader can confound verifier preferences with answer format, we report three checks: a lenient grader (the last number or expression in the final lines when no boxed answer exists; sensitivity only), a blinded human audit, and a re-grading of GSM8K with the revised GSM8K-Platinum labels [34].

3.6. Metrics and controls

For verifier v and condition c, A_N is Best-of-N accuracy and $\Delta_N = A_N^c - A_N^{\mathrm{BF16}}$, computed per problem and averaged. We also report the winner-change rate (the selected candidate differs from BF16's), and its split into harm (a correct BF16 selection becomes wrong) and benefit (the reverse). Two controls bound numerical noise: *bf16-null* rescores in BF16 with a different batch layout, and *bf16-repeat* rescores with the same layout; the latter is bit-identical, the former changes the winner on 7–11% of problems with $|\Delta_{32}| \leq 0.3\,\mathrm{pp}$.

3.7. Statistics and preregistration

Intervals are 95% paired percentile bootstrap intervals over problems (10,000 resamples); p-values are two-sided centred-bootstrap p-values with resolution 10^{-4}. Equivalence is tested by two one-sided tests (TOST) at a $\pm 3\,\mathrm{pp}$ margin, implemented by inverting the percentile interval (simulated size at the margin: 0.056/0.024/0.006 at nominal 0.05/0.025/0.0125).

The main study registered five confirmatory contrasts with Holm correction: c1, Δ_{32} of the 8B ORM at GPTQ-W4 (equivalence); c2, the best development-selected deployment versus the best BF16 deployment under a 6 GB verifier memory ceiling; c3, a calibration method that was not run (kept in the family with $p = 1$); c4, Δ_{32} on MATH (4B and 8B averaged); c5, the growth of the MATH loss from $N = 4$ to $N = 32$. Two follow-up studies were registered after the main results, each with its own family of four tests (Sections 4.5 and 4.6). All deviations are listed in Appendix C.

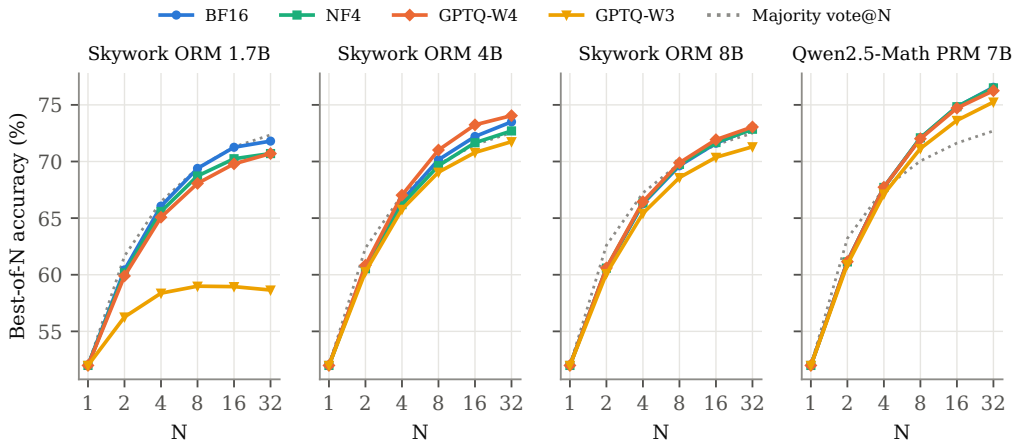


Figure 1: Best-of-N accuracy as a function of the number of candidates N, per verifier and precision (2,000 problems; 200 random subsets per problem for $N < 32$). The dotted line is unweighted majority vote over the same N candidates.

4. Results

4.1. Four-bit verifiers select as well as BF16 verifiers

Table 1 reports Best-of-32 accuracy on the 2,000 evaluation problems. The registered contrast c1 (8B ORM, GPTQ-W4, original 1,200-problem population) is $\Delta_{32} = +0.33$ pp with a 90% interval of $[-0.58, +1.25]$, inside the registered ± 3 pp margin ($p_{\text{TOST}} < 10^{-4}$): **equivalence is supported**. A 3 pp loss would have meant 13% more errors relative to BF16; the interval also lies inside ± 1.5 pp (post hoc). The same holds across the exploratory grid at 4 bits: for the three ORMs and the PRM, GPTQ-W4 and NF4 change A_{32} by between -1.1 and $+0.6$ pp, even though the selected candidate changes on 21–49% of problems. Across budgets $N \in \{1, \ldots, 32\}$ the 4-bit curves track BF16 (Fig. 1).

Three bits is where selection starts to degrade: GPTQ-W3 costs 1.3–

Table 1: Best-of-32 accuracy (%) and change versus BF16 (pp, 95% CI), 2,000 evaluation problems, Qwen2.5-1.5B-Instruct candidates. Winner change: share of problems whose selected candidate differs from BF16's. Majority vote: 72.1–72.7%; oracle pass@32: 86.9%; random candidate: 52.1%.

verifier	precision	A_{32}	Δ_{32} [95% CI]	winner change
Skywork-1.7B	BF16	71.8	–	–
	NF4	70.7	−1.10 [−2.00, −0.25]	34%
	GPTQ-W4	70.7	−1.10 [−2.25, +0.05]	49%
	GPTQ-W3	58.7	−13.15 [−15.05, −11.25]	88%
Skywork-4B	BF16	73.5	–	–
	NF4	72.7	−0.80 [−1.55, −0.10]	24%
	GPTQ-W4	74.1	+0.55 [−0.30, +1.40]	29%
	GPTQ-W3	71.8	−1.75 [−3.05, −0.45]	60%
Skywork-8B	BF16	72.9	–	–
	NF4	72.9	−0.05 [−0.75, +0.65]	21%
	GPTQ-W4	73.1	+0.15 [−0.65, +0.95]	25%
	GPTQ-W3	71.3	−1.60 [−2.75, −0.45]	50%
PRM-7B	BF16	76.6	–	–
	NF4	76.5	−0.05 [−0.65, +0.50]	25%
	GPTQ-W4	76.3	−0.30 [−1.05, +0.45]	33%
	GPTQ-W3	75.3	−1.30 [−2.30, −0.25]	54%

1.8 pp for the 4B, 8B and PRM verifiers and 13.2 pp for the 1.7B ORM, whose selection collapses towards the random-candidate rate. The method matters little at 4 bits for the 4B and 8B ORMs (AWQ-W4: 0.00 and +0.55 pp; round-to-nearest RTN-W4: +0.25 and +0.90 pp; RTN-W8 on the 8B: +0.25 pp,

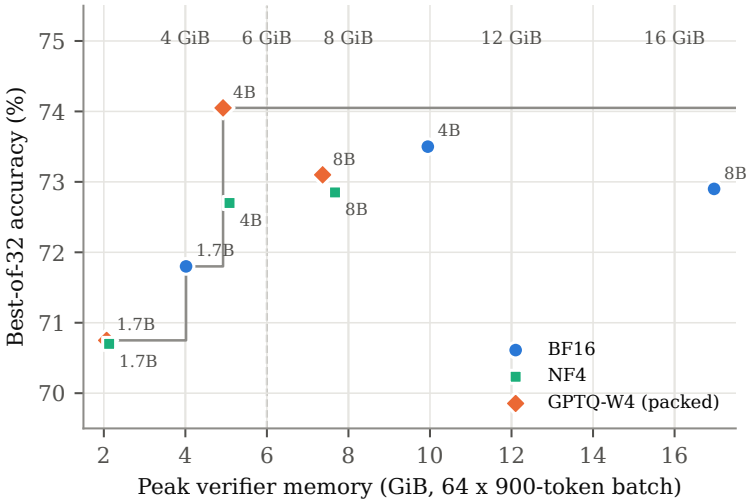


Figure 2: Accuracy–memory frontier of deployed Skywork ORM verifiers (Best-of-32, 2,000 problems). Memory is the measured peak allocation while scoring a fixed workload (64 sequences of 900 tokens, RTX 4090); GPTQ-W4 uses packed 4-bit kernels and NF4 uses `bitsandbytes`. Vertical lines are the registered memory ceilings, applied to the GiB measurements; the dashed one is the primary 6 GiB ceiling of c2. The step line is the best accuracy reachable within each memory budget.

winner change 6%), but the 1.7B ORM is sensitive to it: RTN-W4 costs it 2.8 pp $[-4.0, -1.7]$ against 1.1 pp for GPTQ-W4. Smaller verifiers are less robust to coarser quantization.

4.2. Under a memory budget, quantize the larger verifier

Fig. 2 shows the accuracy–memory frontier of deployed verifiers. At the registered 6 GB ceiling, the configuration selected on development data is the 4B ORM with packed GPTQ-W4 (4.9 GiB peak), against the best BF16 verifier that fits, the 1.7B ORM (4.0 GiB). The registered contrast c2 is $+2.25$ pp $[+1.05, +3.45]$, $p < 10^{-4}$, Holm-adjusted $p = 0.0005$: **supported**.

About a third of this advantage is answer format rather than answer content. With the lenient grader the effect is $+1.55$ pp $[+0.45, +2.65]$. To measure it directly, a human rater graded, blind to arm, verifier and grader verdicts, every Best-of-32 winner without an extractable final answer in either arm (94 responses) plus 30 random controls; a blinded second pass resolved every "unsure" verdict (Appendix B). With these human verdicts replacing the strict grader on all audited winners, c2 is $+1.40$ pp $[+0.30, +2.50]$ (4B GPTQ-W4 75.5% versus 1.7B BF16 74.1%). We therefore report $+1.4$ pp as the format-corrected size of the registered effect. It does not depend on label noise or truncation: with GSM8K labels replaced by GSM8K-Platinum (56 ambiguous problems removed, 8 relabelled) it is $+2.37$ pp, and with length-truncated candidates removed before selection $+2.20$ pp (both under the strict grader).

4.3. Why selection is robust: answer redundancy

Quantization perturbs verifier scores enough to change the selected candidate on a quarter to a half of problems at 4 bits, yet accuracy barely moves. The reason is redundancy in the pool: on most problems many candidates share the selected answer, so a different winner usually carries the same final answer. Harm concentrates on *contested* problems, where the BF16 winner's answer is held by 20–50% of the pool. On these problems (lenient grader), GPTQ-W3 costs the 1.7B ORM 31.0 pp $[-35.9, -25.9]$ ($n = 410$) and the 4B and 8B ORMs 5.4 and 5.7 pp, while GPTQ-W4 costs at most 3.4 pp (1.7B) and 0.0–1.2 pp for the larger models (Fig. 3). The BF16-null control shows the same direction of movement at a much smaller scale, consistent with score noise tipping close decisions. The redundancy bins were fixed after first

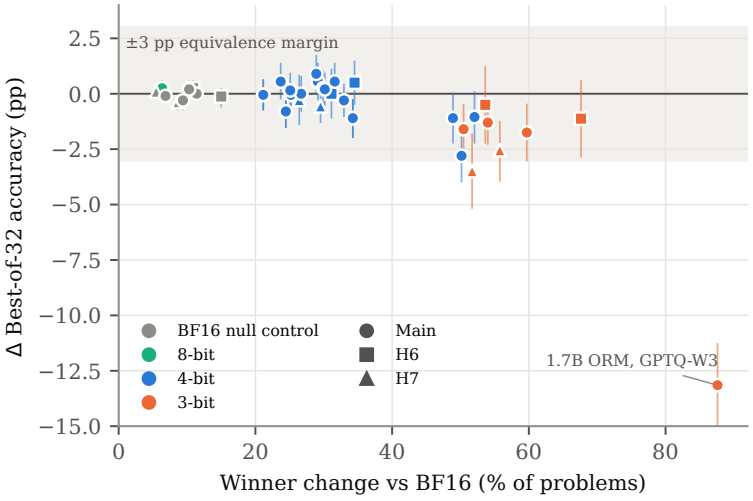


Figure 3: Winner change versus accuracy change at $N = 32$ for every quantized condition and control in the main study (circles) and the two follow-up studies (H6 squares, H7 triangles), with 95% bootstrap intervals. Four-bit conditions change the selected candidate on 20–35% of problems while staying within the ±3 pp equivalence band; 3-bit conditions change about half of the winners and start to lose accuracy.

inspection; this analysis is exploratory.

4.4. Difficulty and budget did not amplify the damage

Development data suggested that quantization damage concentrates on MATH and grows with N; the registered tests on the 1,200 evaluation MATH problems do not support this. c4 (MATH Δ_{32}, 4B and 8B GPTQ-W4 averaged) is -0.63 pp $[-1.38, +0.08]$, Holm $p = 0.40$; c5 (growth from $N = 4$ to $N = 32$) is -0.51 pp $[-1.20, +0.16]$, Holm $p = 0.42$. Both are reported as not supported (inconclusive, not evidence of absence); the intervals bound any such effect at about 1.4 pp. The development signal was a winner's-curse estimate on 200 problems.

4.5. A stronger generator on hard problems (H6)

To test whether robustness survives where selection is harder, we registered a follow-up study before generating any data: Qwen2.5-Math-7B-Instruct [43], 800 MATH Level-5 test problems never used in this work, 32 candidates with up to 2,048 tokens, the 8B ORM and the PRM in BF16 and GPTQ-W4. Its family of four tests asked for equivalence at ± 3 pp (H6a) and for retention of at least half of the verifier's advantage over majority vote (H6b, $R = \frac{1}{2}\,\overline{\text{gain}} + \overline{\Delta} > 0$).

Equivalence is supported for both verifiers: $\Delta_{32} = 0.00$ pp $[-1.13, +1.13]$ for the ORM and $+0.50$ pp $[-0.50, +1.50]$ for the PRM (Holm $p < 10^{-4}$), although quantization changes the selected candidate on 31% and 35% of problems. H6b could not succeed: with this generator, majority vote (76.5%) matched the PRM (76.6%) and beat the ORM (72.8%), so there was no advantage to retain (ORM $R = -1.88$ pp, PRM $+0.56$ pp; not supported). The same holds with the lenient grader and without truncated candidates (19% of candidates reached the limit). Quantization-induced changes moved selections towards the consensus answer: on problems where the BF16 PRM had picked an answer different from majority vote ($n = 131$), its quantized version was right 6.9 pp more often $[+3.1, +11.5]$ (exploratory).

4.6. Value retention where the verifier adds value (H7)

H6 left the value-retention question open because no verifier added value there. We therefore registered a second follow-up study (H7) in the regime where earlier, exploratory data showed a clear advantage over majority vote: the registered weak generator on hard problems. It uses all 1,060 MATH Level-4 and Level-5 test problems not used anywhere else in this work, the

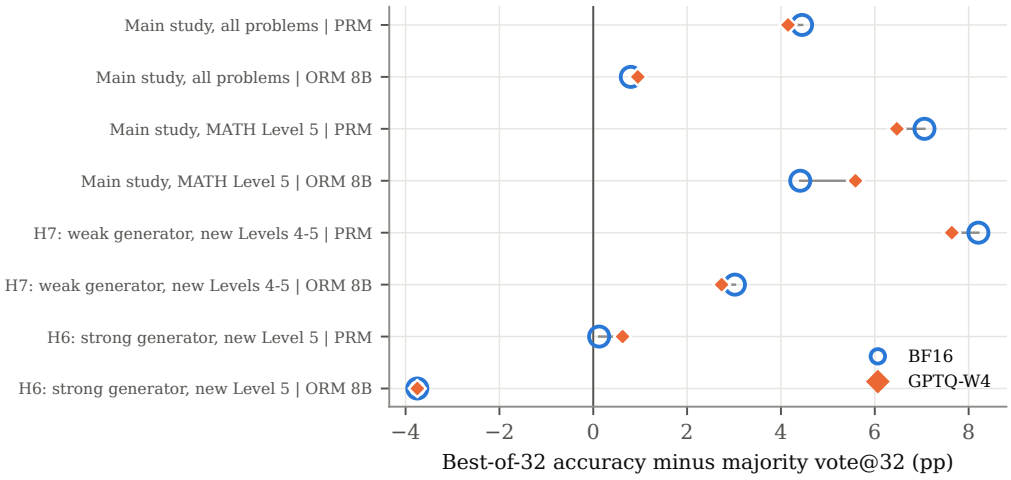


Figure 4: Verifier advantage over majority vote (Best-of-32 accuracy minus majority vote@32) in BF16 (open circles) and with GPTQ-W4 (diamonds), for the 8B ORM and the PRM, across regimes. Main-study rows are exploratory subsets; H6 and H7 are separately preregistered studies on never-used problems.

registered generation settings, and the same verifiers, conditions, tests and analysis code as H6. The regime was chosen with knowledge of the earlier data (selected planning gains of +6.6 and +3.5 pp), so the gains on the new problems could have been smaller; the problems themselves were never seen before.

This time both verifiers beat majority vote in BF16: the PRM by +8.2 pp (97.5% interval $[+5.9, +10.7]$) and the 8B ORM by +3.0 pp $[+0.4, +5.8]$. All four registered hypotheses are supported (Holm over four). Accuracy is equivalent at ±3 pp: $\Delta_{32} = -0.28$ pp $[-1.42, +0.85]$ for the ORM and -0.57 pp $[-1.32, +0.19]$ for the PRM. Quantization keeps at least half of the advantage: for the PRM decisively ($R = +3.54$ pp $[+2.45, +4.72]$, Holm $p < 10^{-4}$; 93% of the advantage retained, 95% interval 84–102%), for the ORM

narrowly ($R = +1.23$ pp, one-sided Holm $p = 0.041$; 91% retained, with a wide interval), as anticipated by its lower registered power. Both conclusions keep their sign under the lenient grader and with truncated candidates removed, the registered condition for stating that value is preserved. Three bits is not safe in this regime: GPTQ-W3 costs 3.5 pp (ORM) and 2.6 pp (PRM) and erases the ORM's advantage (exploratory). Fig. 4 summarises the advantage over majority vote before and after quantization across all regimes.

4.7. Robustness checks

Answer format.. Several quantized conditions appeared to *improve* GSM8K accuracy (e.g. 4B GPTQ-W4 +3.0 pp; +8.0 pp on the Llama pools). A decomposition into format and content shows these are format effects: in BF16 the Skywork ORMs slightly prefer prose answers without a boxed final answer, which the strict grader scores as wrong, and quantization weakens that preference (4B GPTQ-W4: format term +3.0, content term 0.0; lenient +0.4 pp). MATH effects are content effects under both graders. We do not report any quantization "gain".

Second generator.. On 300 problems with Llama-3.2-3B-Instruct candidates (exploratory), GPTQ-W4 changes the PRM's Best-of-32 accuracy by +0.33 pp $[-3.00, +4.00]$ and NF4 by -0.67 pp; for the 4B ORM the strict grader shows +8.0 pp, which shrinks to +2.0 pp $[0.0, +4.3]$ under the lenient grader, again a format effect. With this generator, majority vote (76.3%) beats both verifiers in BF16 (ORM 63.7%, PRM 70.7%).

Labels and truncation.. With GSM8K-Platinum labels c1 is +0.44 pp (c4 and c5 involve MATH only and are unchanged); with truncated candidates removed

before selection, c1 is +0.33, c4 −0.67 and c5 −0.52 pp. No conclusion changes. Both GSM8K problems whose labels our audit had flagged as questionable are among those GSM8K-Platinum removes.

5. Discussion

Deployment guidance.. For reward-model verifiers of 4B parameters and above, 4-bit weight quantization (GPTQ, AWQ or NF4) is a safe default for Best-of-N selection on mathematical problems: across three preregistered settings, two generators, an ORM family and a PRM, the accuracy change stayed within about 1.5 pp and the verifier's advantage over majority vote was largely preserved. Under a fixed memory budget, the freed memory is better spent on a larger quantized verifier than on a smaller full-precision one (c2). Three bits is not a safe default: it is harmless in some regimes (H6) but costs 2–4 pp in others (H7, main study) and is catastrophic for the 1.7B ORM, and the smallest verifier is also sensitive to the quantization method (RTN versus GPTQ).

Why quantization rarely matters, and when it could.. Best-of-N only needs the top of the ranking to keep a correct answer, and candidate pools are redundant: many candidates share the same final answer. Quantization perturbs scores enough to change the selected candidate on a quarter to a third of problems, yet the new winner usually carries the same answer. The risk is concentrated on contested problems, where competing answers have similar support and the verifier's preference is what decides; there, coarse quantization does tip choices (3 bits), and in our H6 and H7 data the perturbation tends to move the selection towards the consensus answer. This

suggests that quantization damage should grow with the share of contested problems and with how much a verifier's decisions depart from the consensus, rather than with nominal difficulty or with N, which is consistent with our unsupported difficulty and budget hypotheses (c4, c5).

How much a verifier is worth.. The question "does quantization destroy the value a verifier adds?" presupposes that the verifier adds value. With the weak generator the Skywork ORMs were within about one point of majority vote overall, and with a strong math generator on hard problems majority vote matched the PRM and beat the ORM by 3.8 pp. Only in a regime selected for verifier advantage (H7) did both verifiers clearly beat majority vote. Practitioners should measure a verifier against self-consistency on their own generator before paying its memory and latency cost.

Evaluating selection.. Three practices changed our conclusions or their size. (i) A strict answer grader turned a reward-model preference for prose answers into apparent quantization "gains" of up to 8 pp; a format/content decomposition and a lenient grader removed them, and exhaustive blinded human grading reduced the registered memory-frontier effect from 2.25 to 1.4 pp. (ii) A fixed-precision null control (same weights, different batch layout) changes 5–15% of winners across our studies; winner-change rates without such a floor overstate instability. (iii) Development-set signals that motivated two registered hypotheses did not replicate on 1,200 evaluation problems; preregistering them made the non-replication visible rather than leaving it in a file drawer.

Limitations.. The ORMs come from a single family (Skywork-Reward-V2, built on Qwen3), the PRM is a Qwen model and the generators are Qwen and Llama models, so family-specific robustness cannot be excluded. Accuracy experiments use simulated quantization; packed 4-bit kernels agreed with it on 98.8–99.2% of selected answers, but we measured memory, not latency or throughput. The tasks are GSM8K and MATH, whose test problems may have been seen during pre-training of the generators and verifiers; contamination affects BF16 and quantized verifiers equally because both score the same frozen pools, but it can change pool redundancy and verifier advantage. Candidates were capped at 1,024 or 2,048 tokens, and 6.5–19% reached the cap, although excluding them does not change any conclusion. The value-retention result is confirmatory for the PRM and only narrowly so for the ORM, and we did not find a strong-generator regime in which the verifiers clearly beat majority vote, so retention of a large advantage under a strong generator remains untested. The analysis plans were fixed in version control before the corresponding data were generated or scored; we did not use an external registry.

6. Conclusion

We asked whether the reward-model verifier in Best-of-N mathematical answer selection can be quantized without losing what it is for. On frozen candidate pools, with analysis plans fixed before the data, the answer is yes at 4 bits for verifiers of 4B parameters and above: GPTQ-W4 keeps Best-of-32 accuracy within a ± 3 pp equivalence margin in the main study and in two follow-up studies with a stronger generator and with new hard problems, and where the verifier beats majority vote it keeps most of that advantage (93%

for the PRM). Under a fixed memory budget a larger quantized verifier is the better choice. Robustness comes from the selection task rather than from score fidelity: quantization changes the selected candidate often, but pools are redundant, so the selected answer rarely changes; 3-bit weights and the smallest verifier are where this breaks down. Two practical caveats travel with these results. Measure a verifier against majority vote on one's own generator, because with a strong generator it may add nothing; and evaluate selection with a grader that is robust to answer format, because a strict grader can turn format preferences into spurious effects of quantization.

CRediT authorship contribution statement

Hector Borobia: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Project administration, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Elies Seguí-Mas:** Writing – review & editing, Supervision. **Guillermina Tormo-Carbó:** Writing – review & editing, Supervision.

Declaration of Generative AI and AI-assisted technologies in the writing process

During the preparation of this work, the authors used Claude Code (Anthropic) to support code development, language editing, and consistency checking. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Code, configuration, frozen problem manifests, analysis plans with their amendments, analysis scripts and figure-generation scripts are available at the project repository: [TODO: https://github.com/hecboar/quantized-verifiers-bon (tag v1.0-submission)]. The candidate pools and all verifier scores are available at [TODO: Hugging Face dataset URL].

References

[1] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, C. Hesse, J. Schulman, Training verifiers to solve math word problems, arXiv preprint arXiv:2110.14168 (2021).

[2] H. Lightman, V. Kosaraju, Y. Burda, H. Edwards, B. Baker, T. Lee, J. Leike, J. Schulman, I. Sutskever, K. Cobbe, Let's verify step by step,

in: The Twelfth International Conference on Learning Representations (ICLR), 2024, arXiv:2305.20050.

[3] C. Snell, J. Lee, K. Xu, A. Kumar, Scaling LLM test-time compute optimally can be more effective than scaling parameters for reasoning, in: The Thirteenth International Conference on Learning Representations (ICLR), 2025, preprint title: "Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters", arXiv:2408.03314.

[4] E. Frantar, S. Ashkboos, T. Hoefler, D. Alistarh, GPTQ: Accurate post-training quantization for generative pre-trained transformers, in: The Eleventh International Conference on Learning Representations (ICLR), 2023, published at ICLR 2023 as "OPTQ: Accurate Quantization for Generative Pre-trained Transformers"; arXiv:2210.17323.

[5] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, S. Han, AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration, in: Proceedings of Machine Learning and Systems, Vol. 6, 2024, arXiv:2306.00978.

[6] T. Dettmers, A. Pagnoni, A. Holtzman, L. Zettlemoyer, QLoRA: Efficient finetuning of quantized LLMs, in: Advances in Neural Information Processing Systems, Vol. 36, 2023, pp. 10088–10115.

[7] L. Gao, J. Schulman, J. Hilton, Scaling laws for reward model overoptimization, in: Proceedings of the 40th International Conference on Machine Learning (ICML), Vol. 202 of Proceedings of Machine Learning Research, PMLR, 2023, pp. 10835–10866.

[8] P. Wang, L. Li, Z. Shao, R. X. Xu, D. Dai, Y. Li, D. Chen, Y. Wu, Z. Sui, Math-Shepherd: Verify and Reinforce LLMs Step-by-step without Human Annotations, arXiv preprint arXiv:2312.08935 (2023). arXiv:2312.08935.

[9] A. Setlur, C. Nagpal, A. Fisch, X. Geng, J. Eisenstein, R. Agarwal, A. Agarwal, J. Berant, A. Kumar, Rewarding Progress: Scaling Automated Process Verifiers for LLM Reasoning, arXiv preprint arXiv:2410.08146 (2024). arXiv:2410.08146.

[10] Z. Zhang, C. Zheng, Y. Wu, B. Zhang, R. Lin, B. Yu, D. Liu, J. Zhou, J. Lin, The lessons of developing process reward models in mathematical reasoning, in: Findings of the Association for Computational Linguistics: ACL 2025, Association for Computational Linguistics, Vienna, Austria, 2025, pp. 10495–10516, arXiv:2501.07301. doi:10.18653/v1/2025.findings-acl.547.

[11] L. Zhang, A. Hosseini, H. Bansal, M. Kazemi, A. Kumar, R. Agarwal, Generative Verifiers: Reward Modeling as Next-Token Prediction, arXiv preprint arXiv:2408.15240 (2024). arXiv:2408.15240.

[12] R. Liu, J. Gao, J. Zhao, K. Zhang, X. Li, B. Qi, W. Ouyang, B. Zhou, Can 1B LLM Surpass 405B LLM? Rethinking Compute-Optimal Test-Time Scaling, arXiv preprint arXiv:2502.06703 (2025). arXiv:2502.06703.

[13] N. Singhi, H. Bansal, A. Hosseini, A. Grover, K.-W. Chang, M. Rohrbach, A. Rohrbach, When To Solve, When To Verify: Compute-Optimal

Problem Solving and Generative Verification for LLM Reasoning, arXiv preprint arXiv:2504.01005 (2025). arXiv:2504.01005.

[14] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, D. Zhou, Self-consistency improves chain of thought reasoning in language models, in: The Eleventh International Conference on Learning Representations (ICLR), 2023, arXiv:2203.11171.

[15] B. Brown, J. Juravsky, R. Ehrlich, R. Clark, Q. V. Le, C. Ré, A. Mirhoseini, Large Language Monkeys: Scaling Inference Compute with Repeated Sampling, arXiv preprint arXiv:2407.21787 (2024). arXiv:2407.21787.

[16] Y. Wu, Z. Sun, S. Li, S. Welleck, Y. Yang, Inference Scaling Laws: An Empirical Analysis of Compute-Optimal Inference for Problem-Solving with Language Models, arXiv preprint arXiv:2408.00724 (2024). arXiv:2408.00724.

[17] L. Yuan, W. Li, H. Chen, G. Cui, N. Ding, K. Zhang, B. Zhou, Z. Liu, H. Peng, Free Process Rewards without Process Labels, arXiv preprint arXiv:2412.01981 (2024). arXiv:2412.01981.

[18] Q. Di, K. Ji, X. Li, H. Zhao, Q. Gu, Best-of-Majority: Minimax-Optimal Strategy for Pass@k Inference Scaling, arXiv preprint arXiv:2510.03199 (2025). arXiv:2510.03199.

[19] J. Kim, E. Ewer, T. Moon, J. Park, D. Papailiopoulos, Not All Bits Are Equal: Scale-Dependent Memory Optimization Strategies for Reasoning Models, arXiv preprint arXiv:2510.10964 (2025). arXiv:2510.10964.

[20] N. Lambert, V. Pyatkin, J. Morrison, L. Miranda, B. Y. Lin, K. Chandu, N. Dziri, S. Kumar, T. Zick, Y. Choi, N. A. Smith, H. Hajishirzi, RewardBench: Evaluating Reward Models for Language Modeling, arXiv preprint arXiv:2403.13787 (2024). arXiv:2403.13787.

[21] C. Zheng, Z. Zhang, B. Zhang, R. Lin, K. Lu, B. Yu, D. Liu, J. Zhou, J. Lin, ProcessBench: Identifying Process Errors in Mathematical Reasoning, arXiv preprint arXiv:2412.06559 (2024). arXiv:2412.06559.

[22] M. Song, Z. Su, X. Qu, J. Zhou, Y. Cheng, PRMBench: A Fine-grained and Challenging Benchmark for Process-Level Reward Models, arXiv preprint arXiv:2501.03124 (2025). arXiv:2501.03124.

[23] T. Dettmers, M. Lewis, Y. Belkada, L. Zettlemoyer, LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale, arXiv preprint arXiv:2208.07339 (2022). arXiv:2208.07339.

[24] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, S. Han, SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models, arXiv preprint arXiv:2211.10438 (2022). arXiv:2211.10438.

[25] T. Dettmers, L. Zettlemoyer, The case for 4-bit precision: k-bit Inference Scaling Laws, arXiv preprint arXiv:2212.09720 (2022). arXiv:2212.09720.

[26] E. Kurtic, A. Marques, S. Pandit, M. Kurtz, D. Alistarh, "Give Me BF16 or Give Me Death"? Accuracy-Performance Trade-Offs in LLM Quantization, arXiv preprint arXiv:2411.02355 (2024). arXiv:2411.02355.

[27] C. Zhu, H. Jiang, J. Liu, Y. Han, K. Zhao, R. Zhang, RotSQ: Joint quantization and sparsification of large language models in rotated space, Neurocomputing 707 (2027) 135039. doi:10.1016/j.neucom.2026.135039.

[28] W. Bang, J. Park, H. Yu, S. Han, K. Bin, K. Lee, BVSR: Bitvector-sum representation for distribution-aware LLM weight quantization, Neurocomputing 709 (2027) 135315. doi:10.1016/j.neucom.2026.135315.

[29] R. Liu, Y. Sun, M. Zhang, H. Bai, X. Yu, T. Yu, C. Yuan, L. Hou, Quantization Hurts Reasoning? An Empirical Study on Quantized Reasoning Models, arXiv preprint arXiv:2504.04823 (2025). arXiv:2504.04823.

[30] Z. Li, Y. Su, R. Yang, C. Xie, Z. Wang, Z. Xie, N. Wong, H. Yang, Quantization Meets Reasoning: Exploring LLM Low-Bit Quantization Degradation for Mathematical Reasoning, arXiv preprint arXiv:2501.03035 (2025). arXiv:2501.03035.

[31] A. Dutta, S. Krishnan, N. Kwatra, R. Ramjee, Accuracy is Not All You Need, arXiv preprint arXiv:2407.09141 (2024). arXiv:2407.09141.

[32] M. Patira, Quantizing the judge: Verdict drift under LLM.int8 and NF4, in: NeurIPS 2026 Workshop on Trustworthy AI for Good (AI4GOOD), 2026, workshop poster.
URL https://openreview.net/forum?id=UOzNwAabR0

[33] H. Zhang, J. Da, D. Lee, V. Robinson, C. Wu, W. Song, T. Zhao, P. Raja, C. Zhuang, D. Slack, Q. Lyu, S. Hendryx, R. Kaplan, M. Lunati, S. Yue, A Careful Examination of Large Language Model Performance

on Grade School Arithmetic, arXiv preprint arXiv:2405.00332 (2024). arXiv:2405.00332.

[34] J. Vendrow, E. Vendrow, S. Beery, A. Madry, Do large language model benchmarks test reliability?, arXiv preprint arXiv:2502.03461 (2025).

[35] A. Hochlehnert, H. Bhatnagar, V. Udandarao, S. Albanie, A. Prabhu, M. Bethge, A Sober Look at Progress in Language Model Reasoning: Pitfalls and Paths to Reproducibility, arXiv preprint arXiv:2504.07086 (2025). arXiv:2504.07086.

[36] S. Biderman, H. Schoelkopf, L. Sutawika, L. Gao, J. Tow, B. Abbasi, A. F. Aji, P. S. Ammanamanchi, S. Black, J. Clive, A. DiPofi, J. Etxaniz, B. Fattori, J. Z. Forde, C. Foster, J. Hsu, M. Jaiswal, W. Y. Lee, H. Li, C. Lovering, N. Muennighoff, E. Pavlick, J. Phang, A. Skowron, S. Tan, X. Tang, K. A. Wang, G. I. Winata, F. Yvon, A. Zou, Lessons from the Trenches on Reproducible Evaluation of Language Models, arXiv preprint arXiv:2405.14782 (2024). arXiv:2405.14782.

[37] E. Miller, Adding Error Bars to Evals: A Statistical Approach to Language Model Evaluations, arXiv preprint arXiv:2411.00640 (2024). arXiv:2411.00640.

[38] J. M. Hofman, A. Chatzimparmpas, A. Sharma, D. J. Watts, J. Hullman, Pre-registration for Predictive Modeling, arXiv preprint arXiv:2311.18807 (2023). arXiv:2311.18807.

[39] D. Hendrycks, C. Burns, S. Kadavath, A. Arora, S. Basart, E. Tang, D. Song, J. Steinhardt, Measuring mathematical problem solving with the

MATH dataset, in: Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks, Vol. 1, 2021, arXiv:2103.03874.

[40] Qwen Team, A. Yang, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Li, D. Liu, F. Huang, H. Wei, et al., Qwen2.5 technical report, arXiv preprint arXiv:2412.15115 (2024).

[41] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, I. Stoica, Efficient memory management for large language model serving with PagedAttention, in: Proceedings of the 29th Symposium on Operating Systems Principles (SOSP), ACM, 2023, pp. 611–626. doi:10.1145/3600006.3613165.

[42] C. Y. Liu, L. Zeng, Y. Xiao, J. He, J. Liu, C. Wang, R. Yan, W. Shen, F. Zhang, J. Xu, Y. Liu, Y. Zhou, Skywork-Reward-V2: Scaling preference data curation via human-AI synergy, in: The Fourteenth International Conference on Learning Representations (ICLR), 2026, arXiv:2507.01352.

[43] A. Yang, B. Zhang, B. Hui, B. Gao, B. Yu, C. Li, D. Liu, J. Tu, J. Zhou, J. Lin, et al., Qwen2.5-Math technical report: Toward mathematical expert model via self-improvement, arXiv preprint arXiv:2409.12122 (2024).

Appendix A. Implementation details

Prompt.. Every generator receives the system prompt "Please reason step by step, and put your final answer within `\boxed{}`." and the problem as

the user turn, through its chat template. For Llama-3.2-3B the instruction is repeated at the end of the user turn (registered deviation; without it 34% of its candidates had no extractable answer).

Verifier inputs.. The Skywork ORMs receive the (problem, response) pair as a user/assistant conversation without system prompt and output a scalar from their reward head. The PRM receives its documented system prompt, the problem, and the response split into steps at blank lines, separated by its step token; the score is the minimum step reward (the last-step reward is stored for sensitivity).

Quantization.. All linear layers inside the transformer blocks are quantized; embeddings, norms and the reward head stay in BF16. GPTQ uses group size 128, symmetric quantization and 1% diagonal damping, calibrated on 128 WikiText-2 sequences of 512 tokens with a fixed seed. Every quantized model is audited by fingerprinting all parameters before and after quantization: every target layer must change (252 of 252 for the 8B ORM, 196 of 196 for the PRM) and no other parameter may change.

PRM rotary buffers.. Under recent versions of `transformers`, Qwen2.5-Math-PRM-7B's remote modelling code is instantiated on the meta device and its non-persistent rotary-embedding buffers come back uninitialised after loading. Scores computed this way were below the random-candidate rate. We recompute the buffers exactly as the model's own initialisation does and check them at every load; all PRM results in this paper use repaired buffers, and the affected scores were discarded (registered deviation).

Compute.. All experiments ran on single cloud GPUs (mostly RTX 4090; some follow-up runs on L40S and other GPUs of at least 24 GB, with BF16 and quantized runs of a verifier always on the same GPU). The total cost of all experiments was about 55 EUR.

Appendix B. Blinded audit of the outcome measure

All 94 unique Best-of-32 winners without an extractable final answer in either arm of the registered memory-frontier contrast (c2), plus 30 random winners with one, were shown to a human rater (one of the authors) with the problem, the reference answer and the full response, rendered with mathematical typesetting, and nothing else; responses that reached the length limit were marked as such and graded as incorrect by a rule fixed beforehand. The rater judged 83 correct, 29 incorrect and 12 unsure. A blinded second pass (same view, key unopened) re-examined every unsure and incorrect verdict, resolved all 12 unsure verdicts as incorrect (9 with wrong reasoning, 3 with correct reasoning cut by the length limit) and confirmed every incorrect one; it also flagged two GSM8K problems with questionable labels, both of which GSM8K-Platinum removes. On the 30 controls the strict grader and the human agree on 29. On the 94 winners without an extractable answer the strict grader scores all as wrong, the human judges 60 correct, and the lenient grader agrees with the human on 85.

Appendix C. Analysis plans and deviations

The main study's plan registered five contrasts before any evaluation score was opened, and an amendment, made after candidate pools existed

but before any verifier had scored them, renamed two hypotheses to match their estimands, fixed populations and test rules, and set a decision rule for the calibration study (c3, not run). The two follow-up studies were each fixed, with their item manifests and analysis code, before their candidates were generated, and their pool hashes were recorded before scoring. The deviations, all documented with dates in the repository, were: the Llama prompt repetition; the PRM rotary-buffer repair and rescoring; the frozen calibration hyper-parameter of the unrun c3 method; exploratory additions after the main results (bit-width dose-response, the 1.7B packed model in the frontier); in H6, the loss and regeneration of the first candidate pool (never inspected) and a scoring split across four GPUs; in H7, the use of GPUs other than the RTX 4090 for lack of availability and the computation of lenient-grader verdicts with time limits equivalent to those on Linux; and corrections to the follow-up analysis script made before H7 data existed, which reproduce all H6 results exactly.